Computation Graphs and Backpropagation

CS 115B – Foundations of NLP II

James Pustejovsky

Computer Science Department
Brandeis University

Spring 2026

Roadmap

1. What is a computation graph?
2. Building graphs from expressions
3. The forward pass — evaluating the graph
4. Local derivatives at each node
5. The backward pass — backpropagation
6. Worked examples (scalar, then vector)
7. Common patterns: addition, multiplication, ReLU, ...
8. Fan-out and gradient accumulation
9. A full neural-network example
10. Connection to PyTorch `autograd`

Part I

What Is a Computation Graph?

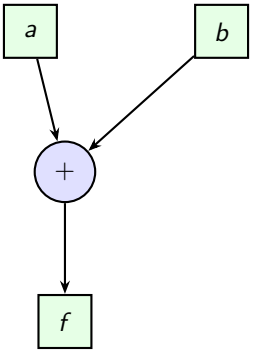
The Core Idea

A **computation graph** is a directed acyclic graph (DAG) that represents a mathematical expression.

- **Leaf nodes** (inputs): variables or constants.
- **Interior nodes** (operations): $+$, $\times$, exp, σ, etc.
- **Edges**: data flow — the output of one node feeds into the next.

Every modern deep-learning framework (PyTorch, JAX, TensorFlow) builds a computation graph behind the scenes.

Simplest Possible Example: $f = a + b$

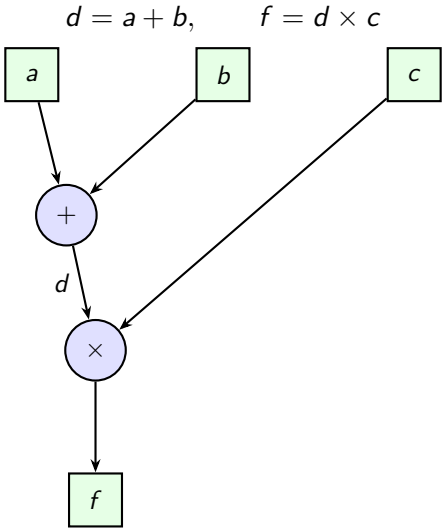


Two inputs flow into an addition node; the output is f.

That's it — computation graphs can be very simple.

A Slightly Bigger Example: $f = (a + b) \times c$

We decompose into **elementary** operations:



Why Decompose into Elementary Operations?

1. Each elementary operation has a **known, simple derivative**.
2. We can compute the derivative of the *whole* expression by chaining these simple derivatives together.
3. This is **backpropagation** — the chain rule applied systematically on a graph.

Think of it as "divide and conquer" for differentiation.

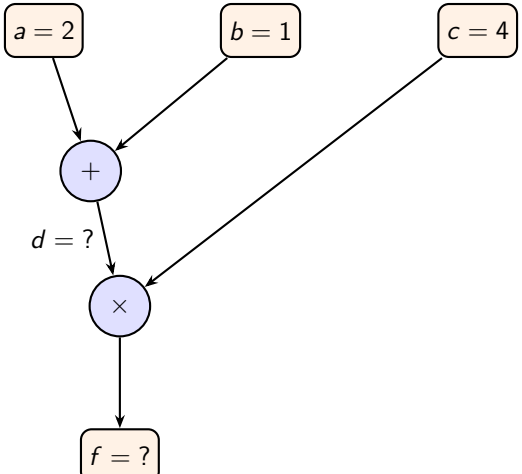
Part II

The Forward Pass

Forward Pass = Evaluate the Expression

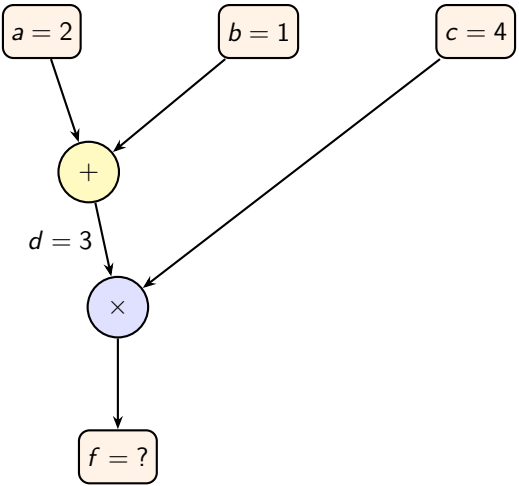
Given concrete input values, we propagate them **forward** through the graph, computing each node's output.

Example: $f = (a + b) \times c$ with $a = 2,\ b = 1,\ c = 4$.



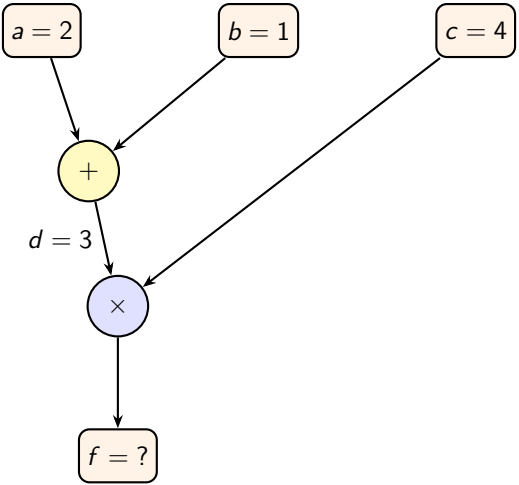
Forward Pass: Step by Step

Step 1: Compute $d = a + b = 2 + 1 = 3$.



Forward Pass: Step by Step

Step 1: Compute $d = a + b = 2 + 1 = 3$.



Step 2: Compute $f = d \times c = 3 \times 4 = 12$.

Part III

Local Derivatives

What is a Local Derivative?

At each operation node, we can compute the derivative of its output with respect to each of its inputs.

These are called **local derivatives** (or **local gradients**).

Addition node: $d = a + b$

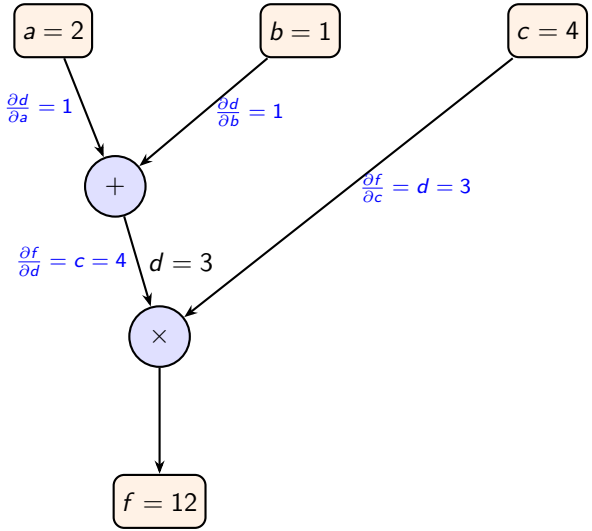
$$\frac{\partial d}{\partial a} = 1, \qquad \frac{\partial d}{\partial b} = 1$$

Multiplication node: $f = d \times c$

$$\frac{\partial f}{\partial d} = c, \qquad \frac{\partial f}{\partial c} = d$$

Addition distributes gradients equally; multiplication "swaps" its inputs.

Annotating the Graph with Local Derivatives



Each edge now carries a local derivative (blue), the "building blocks" for backward pass.

Part IV

The Backward Pass (Backpropagation)

The Goal of the Backward Pass

We want to compute:

$$\frac{\partial f}{\partial a}, \quad \frac{\partial f}{\partial b}, \quad \frac{\partial f}{\partial c}$$

i.e. how the final output f changes when each input is perturbed.

Strategy: Start at the output, work backwards, applying the chain rule at each node.

Backward Pass: Initialize

Seed: The derivative of f with respect to itself is 1.

$$\frac{\partial f}{\partial f} = 1$$

This is always our starting point.

Think of it as: "if f changes by ϵ, then f changes by ϵ." Trivially true.

Backward Pass: Step 1 — Through the Multiply Node

The $\times$ node computes $f = d \times c$.

By the chain rule:

$$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial f} \times \frac{\partial f}{\partial d} = 1 \times c = 1 \times 4 = \mathbf{4}$$

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial f} \times \frac{\partial f}{\partial c} = 1 \times d = 1 \times 3 = \mathbf{3}$$

Backward Pass: Step 1 — Through the Multiply Node

The $\times$ node computes $f = d \times c$.

By the chain rule:

$$\frac{\partial f}{\partial d} = \frac{\partial f}{\partial f} \times \frac{\partial f}{\partial d} = 1 \times c = 1 \times 4 = \mathbf{4}$$

$$\frac{\partial f}{\partial c} = \frac{\partial f}{\partial f} \times \frac{\partial f}{\partial c} = 1 \times d = 1 \times 3 = \mathbf{3}$$

Pattern: at a multiplication node $f = d \times c$, the gradient flowing to d is scaled by c, and vice versa.

Backward Pass: Step 2 — Through the Addition Node

The $+$ node computes $d = a + b$.

The incoming (upstream) gradient is $\frac{\partial f}{\partial d} = 4$.

By the chain rule:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial d} \times \frac{\partial d}{\partial a} = 4 \times 1 = \mathbf{4}$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial d} \times \frac{\partial d}{\partial b} = 4 \times 1 = \mathbf{4}$$

Backward Pass: Step 2 — Through the Addition Node

The $+$ node computes $d = a + b$.

The incoming (upstream) gradient is $\frac{\partial f}{\partial d} = 4$.

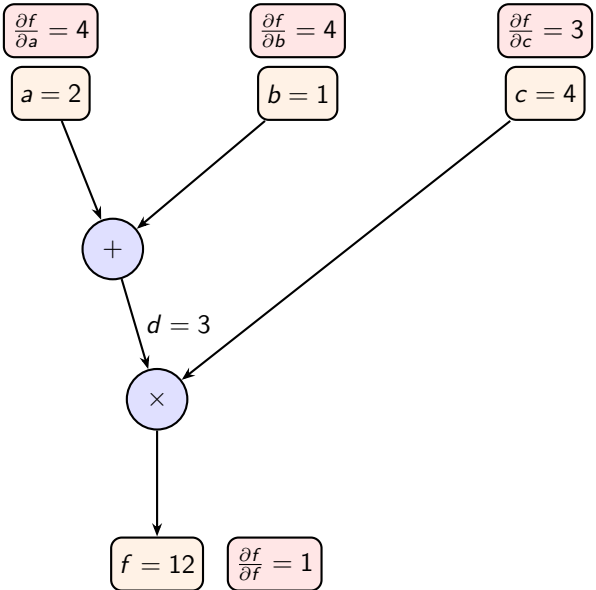
By the chain rule:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial d} \times \frac{\partial d}{\partial a} = 4 \times 1 = \mathbf{4}$$

$$\frac{\partial f}{\partial b} = \frac{\partial f}{\partial d} \times \frac{\partial d}{\partial b} = 4 \times 1 = \mathbf{4}$$

Pattern: an addition node simply **copies** the upstream gradient to both inputs.

Backward Pass: Complete Picture



Let's Verify by Hand

$f = (a + b) \times c = ac + bc$

$$\frac{\partial f}{\partial a} = c = 4 \quad \checkmark \qquad \frac{\partial f}{\partial b} = c = 4 \quad \checkmark \qquad \frac{\partial f}{\partial c} = a + b = 3 \quad \checkmark$$

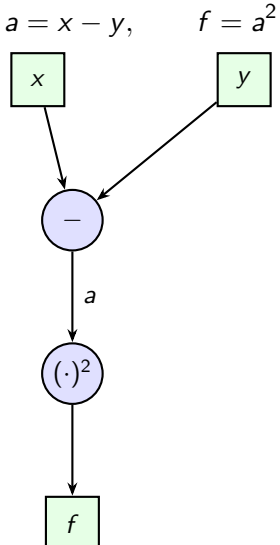
The graph-based approach gave us the same answer as direct differentiation — but the graph approach **scales** to expressions with millions of operations.

Part V

Worked Example: $f = (x - y)^2$

Setting Up the Graph: $f = (x - y)^2$

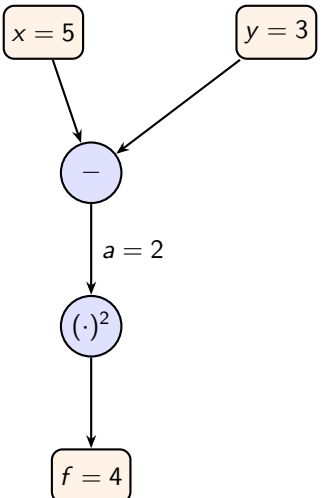
Decompose into elementary ops:



Forward Pass: $x = 5,\ y = 3$

$$a = x - y = 5 - 3 = 2$$

$$f = a^2 = 2^2 = 4$$



Local Derivatives

Subtraction node: $a = x - y$

$$\frac{\partial a}{\partial x} = 1, \qquad \frac{\partial a}{\partial y} = -1$$

Square node: $f = a^2$

$$\frac{\partial f}{\partial a} = 2a$$

At $a = 2$: $\frac{\partial f}{\partial a} = 4$.

Backward Pass

Seed: $\frac{\partial f}{\partial f} = 1$.

Through square node:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial f} \cdot 2a = 1 \cdot 2(2) = 4$$

Backward Pass

Seed: $\frac{\partial f}{\partial f} = 1$.

Through square node:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial f} \cdot 2a = 1 \cdot 2(2) = 4$$

Through subtraction node:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x} = 4 \cdot 1 = \mathbf{4}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial y} = 4 \cdot (-1) = \mathbf{-4}$$

Backward Pass

Seed: $\frac{\partial f}{\partial f} = 1$.

Through square node:

$$\frac{\partial f}{\partial a} = \frac{\partial f}{\partial f} \cdot 2a = 1 \cdot 2(2) = 4$$

Through subtraction node:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x} = 4 \cdot 1 = \mathbf{4}$$

$$\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial y} = 4 \cdot (-1) = \mathbf{-4}$$

Verify: $f = (x - y)^2 \Rightarrow \frac{\partial f}{\partial x} = 2(x - y) = 2(2) = 4$. ✓

Part VI

Worked Example: $f = \sigma(wx + b)$

A Single Neuron

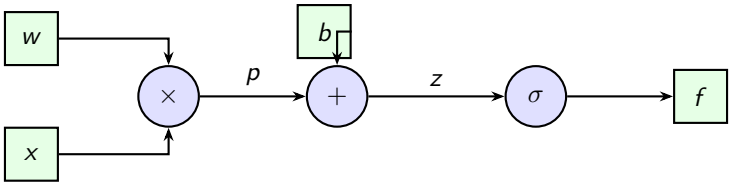
The computation of one neuron with sigmoid activation:

$$f = \sigma(wx + b) \qquad \text{where} \quad \sigma(z) = \frac{1}{1 + e^{-z}}$$

Decompose:

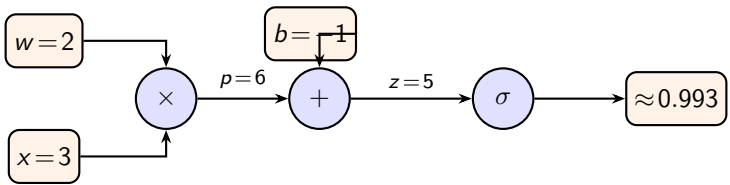
$$p = w \cdot x \qquad \text{(multiply)}$$
$$z = p + b \qquad \text{(add bias)}$$
$$f = \sigma(z) \qquad \text{(activation)}$$

The Graph for a Single Neuron



Forward Pass: $w = 2,\ x = 3,\ b = -1$

$$p = w \cdot x = 2 \cdot 3 = 6$$
$$z = p + b = 6 + (-1) = 5$$
$$f = \sigma(5) = \frac{1}{1 + e^{-5}} \approx 0.9933$$



Local Derivatives for the Neuron

Node	Operation	Local deriv.	Value
$\times$	$p = wx$	$\frac{\partial p}{\partial w} = x = 3, \quad \frac{\partial p}{\partial x} = w = 2$	
$+$	$z = p + b$	$\frac{\partial z}{\partial p} = 1, \quad \frac{\partial z}{\partial b} = 1$	
σ	$f = \sigma(z)$	$\frac{\partial f}{\partial z} = f(1 - f) \approx 0.0066$	

Recall: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, a beautiful self-referential derivative.

Backward Pass: Neuron (1/3)

Seed: $\frac{\partial f}{\partial f} = 1$.

Through σ node:

$$\frac{\partial f}{\partial z} = 1 \cdot \sigma(z)(1 - \sigma(z)) = 0.9933 \times 0.0067 \approx 0.0066$$

The upstream gradient reaching z is approximately 0.0066.

Notice how the sigmoid "squashes" the gradient when the neuron is saturated.

Backward Pass: Neuron (2/3)

Through $+$ **node**: the gradient passes through unchanged.

$$\frac{\partial f}{\partial p} = 0.0066 \times 1 = 0.0066$$

$$\frac{\partial f}{\partial b} = 0.0066 \times 1 = \mathbf{0.0066}$$

The gradient with respect to the bias is always equal to the upstream gradient.

Backward Pass: Neuron (3/3)

Through $\times$ node:

$$\frac{\partial f}{\partial w} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial w} = 0.0066 \times x = 0.0066 \times 3 = \mathbf{0.0198}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial x} = 0.0066 \times w = 0.0066 \times 2 = \mathbf{0.0132}$$

Backward Pass: Neuron (3/3)

Through $\times$ node:

$$\frac{\partial f}{\partial w} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial w} = 0.0066 \times x = 0.0066 \times 3 = \mathbf{0.0198}$$

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial p} \cdot \frac{\partial p}{\partial x} = 0.0066 \times w = 0.0066 \times 2 = \mathbf{0.0132}$$

Summary of gradients:

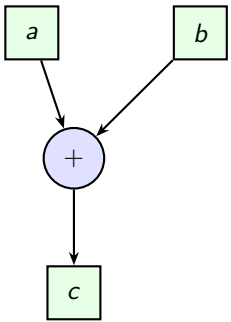
$$\frac{\partial f}{\partial w} \approx 0.020, \quad \frac{\partial f}{\partial x} \approx 0.013, \quad \frac{\partial f}{\partial b} \approx 0.007$$

These are the numbers a gradient descent step would use.

Part VII

Common Node Patterns

The Addition Gate



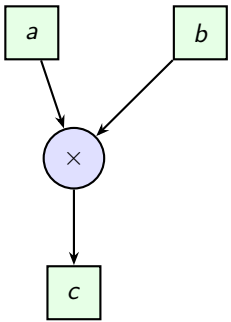
$c = a + b$

Local derivatives:

$$\frac{\partial c}{\partial a} = 1, \quad \frac{\partial c}{\partial b} = 1$$

Behavior: Addition is a **gradient distributor**. It copies the upstream gradient equally to both inputs.

The Multiplication Gate



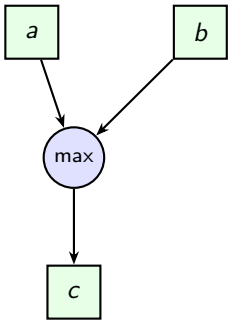
$c = a \times b$

Local derivatives:

$$\frac{\partial c}{\partial a} = b, \quad \frac{\partial c}{\partial b} = a$$

Behavior: Multiplication is a **gradient switcher**. The gradient flowing to a is scaled by b's value, and vice versa.

The Max Gate



$c = \max(a, b)$

Local derivatives:

$$\frac{\partial c}{\partial a} = \begin{cases} 1 & \text{if } a > b \\ 0 & \text{otherwise} \end{cases}$$

Behavior: Max is a **gradient router**. It sends the full gradient to the larger input and zero to the other.

The ReLU Gate

ReLU is just $\max(0, x)$:

$$\text{ReLU}(x) = \max(0, x), \qquad \frac{\partial \text{ReLU}}{\partial x} = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \end{cases}$$

Behavior: ReLU is a **gradient gate**.

- If $x > 0$: gradient passes through unchanged.
- If $x \leq 0$: gradient is **killed** (set to zero).

This is why "dead ReLU" neurons are a problem — once a ReLU goes negative, it receives zero gradient and can never recover.

Summary of Gate Behaviors

Gate	Forward	Gradient behavior
Addition	$a + b$	*Distributes*: copies gradient to both
Multiply	$a \times b$	*Switches*: scales by other input
Max	$\max(a, b)$	*Routes*: sends gradient to winner
ReLU	$\max(0, x)$	*Gates*: passes or kills gradient
Sigmoid	$\sigma(x)$	*Squashes*: scales by $\sigma(1 - \sigma)$

These intuitions help you reason about gradient flow without doing any math.

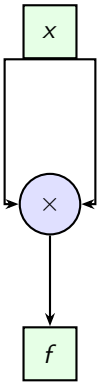
Part VIII

Fan-Out and Gradient Accumulation

What Happens When a Variable is Used Twice?

Consider $f = x \cdot x = x^2$.

The variable x **fans out** to both inputs of the multiply node.



How do we handle the backward pass?

The Multivariate Chain Rule: Gradients Add Up

When a variable x contributes to f through **multiple paths**, we **sum** the gradients from each path.

$f = x \times x$. The $\times$ node gives:

- Gradient to left input: $\frac{\partial f}{\partial \text{left}} = x$ (right input value)
- Gradient to right input: $\frac{\partial f}{\partial \text{right}} = x$ (left input value)

The Multivariate Chain Rule: Gradients Add Up

When a variable x contributes to f through **multiple paths**, we **sum** the gradients from each path.

$f = x \times x$. The $\times$ node gives:

- Gradient to left input: $\frac{\partial f}{\partial \text{left}} = x$ (right input value)
- Gradient to right input: $\frac{\partial f}{\partial \text{right}} = x$ (left input value)

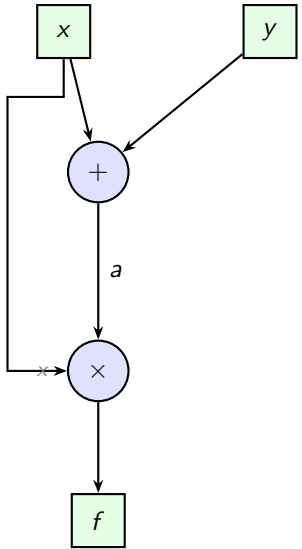
Both paths lead back to x, so we **sum**:

$$\frac{\partial f}{\partial x} = x + x = 2x$$

Which is exactly the derivative of x^2. ✓

Worked Example: $f = (x + y) \cdot x$

Decompose: $a = x + y, \quad f = a \cdot x.$



Forward Pass: $x = 3,\ y = 2$

$$a = x + y = 3 + 2 = 5, \qquad f = a \cdot x = 5 \cdot 3 = 15$$

Forward Pass: $x = 3,\ y = 2$

$$a = x + y = 3 + 2 = 5, \qquad f = a \cdot x = 5 \cdot 3 = 15$$

Backward pass: Seed $\frac{\partial f}{\partial f} = 1$.

Through $\times$ node ($f = a \cdot x$):

- $\frac{\partial f}{\partial a} = x = 3$
- Gradient to x via this path: $a = 5$

Forward Pass: $x = 3,\ y = 2$

$$a = x + y = 3 + 2 = 5, \qquad f = a \cdot x = 5 \cdot 3 = 15$$

Backward pass: Seed $\frac{\partial f}{\partial f} = 1$.

Through $\times$ node ($f = a \cdot x$):

- $\frac{\partial f}{\partial a} = x = 3$
- Gradient to x via this path: $a = 5$

Through $+$ node ($a = x + y$):

- $\frac{\partial f}{\partial x}\Big|_{\text{via } a} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x} = 3 \cdot 1 = 3$
- $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial y} = 3 \cdot 1 = 3$

Forward Pass: $x = 3,\ y = 2$

$$a = x + y = 3 + 2 = 5, \qquad f = a \cdot x = 5 \cdot 3 = 15$$

Backward pass: Seed $\frac{\partial f}{\partial f} = 1$.

Through $\times$ node ($f = a \cdot x$):

- $\frac{\partial f}{\partial a} = x = 3$
- Gradient to x via this path: $a = 5$

Through $+$ node ($a = x + y$):

- $\frac{\partial f}{\partial x}\big|_{\text{via } a} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial x} = 3 \cdot 1 = 3$
- $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial a} \cdot \frac{\partial a}{\partial y} = 3 \cdot 1 = 3$

Total gradient at x: $5 + 3 = \mathbf{8}$. (Verify: $f = x^2 + xy$, $\frac{\partial f}{\partial x} = 2x + y = 8$. ✓)

The Fan-Out Rule

When a variable feeds into multiple downstream nodes, its gradient is the **sum** of the gradients flowing back from each path.

This is the computation-graph version of the **multivariable chain rule**:

$$\frac{\partial f}{\partial x} = \sum_{\text{paths}} \frac{\partial f}{\partial \text{child}_i} \cdot \frac{\partial \text{child}_i}{\partial x}$$

PyTorch handles this automatically with `.backward()` — gradients are accumulated at each tensor.

Part IX

Worked Example: Softmax + Cross-Entropy

Setup: Binary Classification

Suppose we have a single logit z and a binary label $y \in \{0, 1\}$.

Prediction: $\hat{y} = \sigma(z)$
Loss: binary cross-entropy

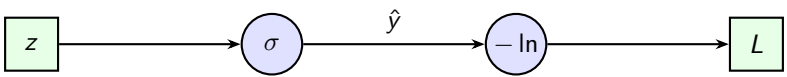
$$L = -\big[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})\big]$$

Let's build the computation graph and backpropagate through it.

The Computation Graph

Decompose (for $y = 1$ to keep it concrete):

$$\hat{y} = \sigma(z)$$
$$L = -\ln(\hat{y})$$



Forward Pass: $z = 1.0$, $y = 1$

$$\hat{y} = \sigma(1.0) = \frac{1}{1 + e^{-1}} \approx 0.731$$

$$L = -\ln(0.731) \approx 0.313$$

The loss is 0.313 — the model is fairly confident but not perfect.

Backward Pass

Seed: $\frac{\partial L}{\partial L} = 1$.

Through $-\ln$ **node**: $L = -\ln(\hat{y})$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{1}{\hat{y}} = -\frac{1}{0.731} \approx -1.369$$

Backward Pass

Seed: $\frac{\partial L}{\partial L} = 1$.

Through $-\ln$ **node**: $L = -\ln(\hat{y})$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{1}{\hat{y}} = -\frac{1}{0.731} \approx -1.369$$

Through σ **node**: $\hat{y} = \sigma(z)$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \hat{y}(1-\hat{y}) = (-1.369)(0.731)(0.269) \approx -0.269$$

Backward Pass

Seed: $\frac{\partial L}{\partial L} = 1$.

Through $-\ln$ **node**: $L = -\ln(\hat{y})$

$$\frac{\partial L}{\partial \hat{y}} = -\frac{1}{\hat{y}} = -\frac{1}{0.731} \approx -1.369$$

Through σ **node**: $\hat{y} = \sigma(z)$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial \hat{y}} \cdot \hat{y}(1-\hat{y}) = (-1.369)(0.731)(0.269) \approx -0.269$$

Elegant simplification: For $y = 1$,

$$\frac{\partial L}{\partial z} = \hat{y} - y = 0.731 - 1 = -0.269 \quad \checkmark$$

The sigmoid + cross-entropy gradient is simply $(\hat{y} - y)$.

Why This Matters for NLP

The sigmoid/softmax + cross-entropy gradient $(\hat{y} - y)$ appears everywhere:

- Binary text classification (spam, sentiment)
- Each output unit in multi-label classification
- The output layer of language models (softmax + CE generalizes this)

The computation graph perspective shows us *why* this simplification happens — cancellation of terms through the chain rule.

Part X

Moving to Vectors and Matrices

From Scalars to Vectors

So far, every node carried a scalar value.

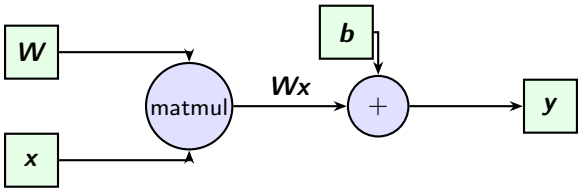
In practice, neural network layers operate on **vectors** and **matrices**.

Good news: The same graph structure and backward pass algorithm apply.
Change: Local derivatives become **Jacobian matrices**, and the chain rule becomes matrix multiplication.

(See companion lecture: *Jacobians and the Vector-Jacobian Product*.)

Example: A Linear Layer

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b}, \qquad \boldsymbol{W} \in \mathbb{R}^{m \times n},\ \boldsymbol{x} \in \mathbb{R}^{n},\ \boldsymbol{b} \in \mathbb{R}^{m}$$



Backward Pass for the Linear Layer

Given upstream gradient $\boldsymbol{v} = \frac{\partial L}{\partial \boldsymbol{y}} \in \mathbb{R}^m$:

Through $+$ node: gradient passes through to both inputs.

$$\frac{\partial L}{\partial \boldsymbol{b}} = \boldsymbol{v}$$

Through matmul node:

$$\frac{\partial L}{\partial \boldsymbol{x}} = \boldsymbol{W}^\top \boldsymbol{v} \qquad \in \mathbb{R}^n$$

$$\frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{v}\,\boldsymbol{x}^\top \qquad \in \mathbb{R}^{m \times n}$$

These are the VJP rules for the linear layer — the most common operation in deep learning.

Worked Example: Linear Layer

Let $\boldsymbol{W} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $\boldsymbol{x} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $\boldsymbol{b} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.

Forward:

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b} = \begin{pmatrix} 1-2 \\ 3-4 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$$

Worked Example: Linear Layer

Let $\boldsymbol{W} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$, $\boldsymbol{x} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$, $\boldsymbol{b} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.

Forward:

$$\boldsymbol{y} = \boldsymbol{W}\boldsymbol{x} + \boldsymbol{b} = \begin{pmatrix} 1-2 \\ 3-4 \end{pmatrix} = \begin{pmatrix} -1 \\ -1 \end{pmatrix}$$

Backward with upstream gradient $\boldsymbol{v} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$:

$$\frac{\partial L}{\partial \boldsymbol{x}} = \boldsymbol{W}^\top \boldsymbol{v} = \begin{pmatrix} 1 & 3 \\ 2 & 4 \end{pmatrix} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 4 \\ 6 \end{pmatrix}$$

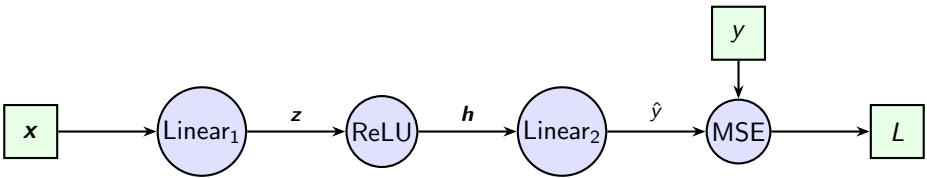
$$\frac{\partial L}{\partial \boldsymbol{W}} = \boldsymbol{v}\,\boldsymbol{x}^\top = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \begin{pmatrix} 1 & -1 \end{pmatrix} = \begin{pmatrix} 1 & -1 \\ 1 & -1 \end{pmatrix}$$

Part XI

Putting It All Together: A Two-Layer Network

A Two-Layer Network

$$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1), \qquad \hat{y} = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2, \qquad L = (\hat{y} - y)^2$$



Each box is itself a sub-graph we already know how to differentiate.

Forward Pass with Concrete Numbers

Let $\boldsymbol{x} = (1, -1)^\top$, $y = 1$, and simple weights:

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \ \boldsymbol{b}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \ \boldsymbol{w}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \ b_2 = 0$$

Forward Pass with Concrete Numbers

Let $\boldsymbol{x} = (1, -1)^\top$, $y = 1$, and simple weights:

$$\boldsymbol{W}_1 = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \ \boldsymbol{b}_1 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \ \boldsymbol{w}_2 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \ b_2 = 0$$

$$\begin{aligned} \boldsymbol{z} &= \boldsymbol{W}_1 \boldsymbol{x} + \boldsymbol{b}_1 = (1, -1)^\top \\ \boldsymbol{h} &= \mathrm{ReLU}(\boldsymbol{z}) = (1, 0)^\top \quad \text{(second component killed)} \\ \hat{y} &= \boldsymbol{w}_2^\top \boldsymbol{h} + b_2 = 1 + 0 = 1 \\ L &= (\hat{y} - y)^2 = (1 - 1)^2 = 0 \end{aligned}$$

The loss is zero — the network happens to produce the right answer.

Now Let's Change $y = 0$ So There's a Nonzero Loss

Same weights, but target $y = 0$:

$$\hat{y} = 1 \quad \text{(same forward pass)}$$
$$L = (1 - 0)^2 = 1$$

Now Let's Change $y = 0$ So There's a Nonzero Loss

Same weights, but target $y = 0$:

$$\hat{y} = 1 \quad \text{(same forward pass)}$$
$$L = (1 - 0)^2 = 1$$

Backward pass: Seed $\frac{\partial L}{\partial L} = 1$.

Through MSE: $L = (\hat{y} - y)^2$

$$\frac{\partial L}{\partial \hat{y}} = 2(\hat{y} - y) = 2(1 - 0) = 2$$

Backward Through Linear$_2$

$\hat{y} = \boldsymbol{w}_2^\top \boldsymbol{h} + b_2$ with upstream gradient $\frac{\partial L}{\partial \hat{y}} = 2$.

$$\frac{\partial L}{\partial \boldsymbol{h}} = 2 \cdot \boldsymbol{w}_2 = 2 \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 2 \\ 2 \end{pmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{w}_2} = 2 \cdot \boldsymbol{h} = 2 \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

$$\frac{\partial L}{\partial b_2} = 2$$

Backward Through ReLU

$\boldsymbol{h} = \mathrm{ReLU}(\boldsymbol{z})$ with upstream gradient $\frac{\partial L}{\partial \boldsymbol{h}} = (2, 2)^\top$.

ReLU backward: multiply elementwise by $\mathbf{1}[z_i > 0]$.

$$\frac{\partial L}{\partial \boldsymbol{z}} = \frac{\partial L}{\partial \boldsymbol{h}} \odot \mathbf{1}[\boldsymbol{z} > 0] = \begin{pmatrix} 2 \\ 2 \end{pmatrix} \odot \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

The gradient for the second component is killed because $z_2 = -1 < 0$.

Backward Through Linear$_1$

$\boldsymbol{z} = \boldsymbol{W}_1\boldsymbol{x} + \boldsymbol{b}_1$ with upstream gradient $\frac{\partial L}{\partial \boldsymbol{z}} = (2, 0)^\top$.

$$\frac{\partial L}{\partial \boldsymbol{x}} = \boldsymbol{W}_1^\top \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 2 \\ 0 \end{pmatrix} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{W}_1} = \begin{pmatrix} 2 \\ 0 \end{pmatrix} \begin{pmatrix} 1 & -1 \end{pmatrix} = \begin{pmatrix} 2 & -2 \\ 0 & 0 \end{pmatrix}$$

$$\frac{\partial L}{\partial \boldsymbol{b}_1} = \begin{pmatrix} 2 \\ 0 \end{pmatrix}$$

Gradient Summary for the Two-Layer Network

Parameter	Gradient
$\boldsymbol{W}_1$	$\begin{pmatrix} 2 & -2 \\ 0 & 0 \end{pmatrix}$
$\boldsymbol{b}_1$	$\begin{pmatrix} 2 \\ 0 \end{pmatrix}$
$\boldsymbol{w}_2$	$\begin{pmatrix} 2 \\ 0 \end{pmatrix}$
b_2	2

Notice: all second-row/component gradients are zero because ReLU killed that path. The gradient tells us: only the first hidden unit matters for reducing this loss.